

Energy efficient mixed task handling on real-time embedded systems using FreeRTOS

Deepak Ramegowda, Man Lin *

St. Francis Xavier University, Canada

ARTICLE INFO

Keywords:

DVFS
Power management
FreeRTOS

ABSTRACT

With the increased use of embedded devices in Industrial electronics, like relays, battery life has gained more and more attention. Modern processors can use Dynamic Voltage Frequency Scaling (DVFS) techniques for energy reduction and temperature control. However, DVFS is not well supported by systems running with Real-Time operating systems like FreeRTOS, which is a widely used real-time operation system (RTOS) in the industry. Furthermore, energy-efficient DVFS techniques for Mixed Tasksets (including periodic and aperiodic tasks) are hardly investigated. This paper extends the classic well-known DVFS technique Cycle Conserving algorithm to handle Mixed Taskset (CCMT algorithm) and implements it on a real-time embedded platform powered by FreeRTOS. We describe our experience implementing CCMT on a real platform with limited DVFS and corresponding scheduler support. Results show that we can successfully apply CCMT to handle aperiodic requests while meeting the deadlines of the periodic tasks and saving energy on the FreeRTOS platform. The algorithm is tested on an ARM Cortex-M7 processor integrated with frequency scaling and power management. Over 5% to 10% energy savings can be achieved for a standard real-time scheduling mechanism without penalty in application throughput.

1. Introduction

Power consumption plays a vital role in real-time embedded systems. It is significant to design portable embedded systems that deliver higher performance with longer battery life. Among the various components like the display, Bluetooth, and other peripherals in a computational device, the processor is a significant power consumer whose performance is directly related to its energy dissipation (see Fig. 1).

The power consumption of a silicon-based CMOS processor can be divided mainly into dynamic power and leakage power. Dynamic power (P_{dp}) is consumed during the execution of instructions. It is determined by a silicon process technology and is approximately proportional to the operating frequency (f), and the square of the operating voltage (V) [1]. In contrast, leakage power (P_{lp}) is mainly determined by silicon process technology. It is caused by leakage current, which flows in the circuit even while instructions are not being executed [2]. If a task has an execution time c_n at voltage V_n and frequency f_n , its normalized CPU energy consumption can be estimated as follows [3].

$$E_n = \lambda \times V_n^2 \times f_n \times c_n + \rho \times V_n \times c_n \quad (1)$$

where, “ λ ” and “ ρ ” are the percentage contribution of the dynamic and the leakage power to the total power consumption, respectively.

As shown above, frequency affects power consumption. This is why dynamic voltage/frequency scaling (DVFS) is widely used to lower power consumption [1–7].

FreeRTOS is a leading, real-time operating system for microcontrollers and small microprocessors developed in partnership with the world's leading chip companies that include a kernel and a growing set of libraries suitable for use across all industry sectors. However, energy-saving with a DVFS technique is not well supported by systems running with FreeRTOS.

This work's main objective is to study the implementation and evaluation of a DVFS technique for applications running on the FreeRTOS platform. We chose a well-known simple dynamic DVFS technique: Cycle Conservative (CC) [6] to start with. Real-time systems may contain both periodic and aperiodic requests. To handle both periodic and aperiodic requests, we extend CC to Cycle Conserving Mixed Task (CCMT) and implement it in a FreeRTOS platform. Though the cycle conserving algorithm is simple when implementing it in a simulator, it is challenging when implementing it on a real platform with limited support of DVFS and corresponding scheduler support. CCMT adopts the EDF scheduling policy, which is not supported by the scheduler of FreeRTOS. We integrate the ESFree Library [9] to support EDF scheduling. To the best of our knowledge, we are the first to experiment

* Corresponding author.

E-mail address: m.lin@stfx.ca (M. Lin).

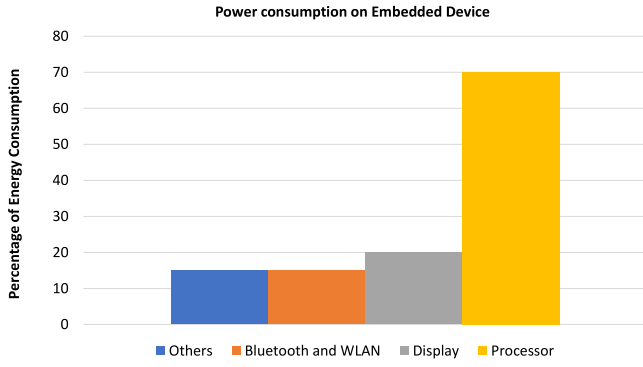


Fig. 1. Power consumption on embedded platform [8].

Table 1
Sample taskset with period and WCET.

Task No. (T_i)	Period (P_i)	WCET (W_i)
T_1	$P_1 = 4$	$W_1 = 1$
T_2	$P_2 = 6$	$W_2 = 2$
T_3	$P_3 = 12$	$W_3 = 3$

Table 2
Actual execution time of a periodic taskset.

Actual execution time ($A_{i,j}$)	J_1	J_2	J_3
T_1	$A_{1,1} = 0.5$	$A_{1,2} = 1.0$	$A_{1,3} = 1.0$
T_2	$A_{2,1} = 2.0$	$A_{2,2} = 1.5$	$A_{2,3} = 2.0$
T_3	$A_{3,1} = 3.0$	$A_{3,2} = 2.5$	$A_{3,3} = 3.0$

dynamic frequency scaling approach on the FreeRTOS framework with a real embedded device and to analyze the real-time constraints of a mixed task on such a platform. To summarize, we work on real hardware devices considering the above factors on FreeRTOS for DVFS task scheduling while focusing on system energy saving.

2. Cycle conserving DVFS technique for periodic tasks

The Cycle Conservative (CC) algorithm is an effective DVFS method that adjusts the frequency dynamically based on the dynamic slack of tasks [6]. Next, we show the working of the CC technique.

CC uses Earliest Deadline First (EDF) scheduling policy [6,10,11]. EDF is a dynamic priority scheduling policy that gives the highest priority to the periodic task with the least deadline. EDF can guarantee the schedulability of periodic tasks if the CPU utilization is less than 100 percent [12].

Let us first consider Table 1 that shows the periodic taskset composed of the period P_i and the worst-case execution time W_i for each task T_i . We assume the deadline of the task is equal to the period.

The actual execution time of a task is normally less than the worst-case execution time. Table 2 shows the actual execution time of the different instances of the tasks (each instance is called a job) [1]: $A_{i,j}$ means actual execution time for task job instance J_j of task T_i .

When a task is released, the CC approach assumes that the task will take its WCET to execute and sets the frequency accordingly. On completion, if the actual execution time is less than the worst-case execution time, then the extra unused cycles can be transferred to the remaining tasks. As the remaining tasks now get more cycles than they require to execute, the frequency can now be scaled down. The initial frequency (when $t = 0$) is set to 0.8333. This is derived as $f = W_1/P_1 + W_2/P_2 + W_3/P_3 = 1/4 + 2/6 + 3/12 = 0.833$. Note that the frequency in this paper is normalized to [0,1]. T_1 is chosen to execute as the deadline of T_1 is 4, which is the earliest. The first instance of T_1 is J_1 , whose actual execution is $A_{1,1} = 0.5$. Thus J_1 finishes at $t = 0.5/0.833 = 0.6$. At $t = 0.6$ (when J_1 finishes execution),

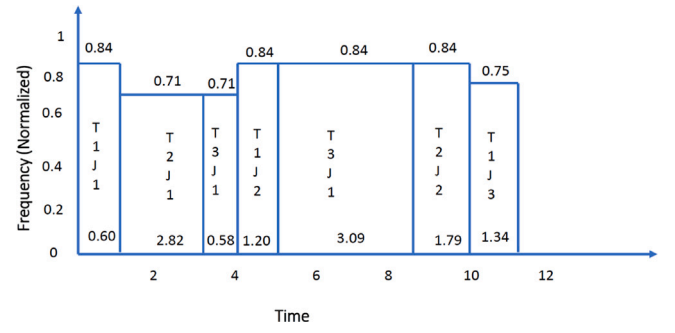


Fig. 2. CC algorithm implementation [1].

the schedule calculate the new frequency. The new frequency is set to $0.71 = A_{1,1}/P_1 + W_2/P_2 + W_3/P_3 = 0.5/4 + 2/6 + 3/12$. The scaling trace for the above example can be found in Fig. 2. More details can be found in [1,13].

3. Mixed taskset model

The mixed taskset model includes the periodic taskset model (with an example shown in Table 1 and Table 2), and the aperiodic tasks model. To handle aperiodic tasks, a periodic server will be created [14, 15]. A server can be described as below [15].

$$S_t = \{P_s, W_s\}$$

where S_t represents the server task in which P_s is the server period, and W_s is the server budget. Upon the arrival of any aperiodic request, the budget allocated by the server task will be consumed. The total utilization is now updated for a Mixed Taskset as [15]:

$$U = \sum_{\forall i \in T} \frac{w_i}{P_i} + \frac{w_s}{P_s} \quad (2)$$

where P_s is the server period, W_s is the server budget, p_i is the period, and w_i is the Worst Case Execution Time (WCET) for a periodic task.

An aperiodic request is defined as: $A_t = \{A_1, A_2, \dots, A_n\}$ where n represents the number of aperiodic tasks in the taskset A_t . The Mixed Taskset queue will be updated based on the arrival time of the aperiodic task. Based on the earliest deadline priority, the aperiodic request is served from the Mixed Taskset queue. Therefore, the response time is based purely on the EDF priority (server period). The response time R_t can be calculated as follows:

$$R_t = C_m - A_r,$$

where A_r represents the arrival time of an aperiodic request, and C_m represents the completion time. Server period gets scheduled based on the EDF policy [14].

4. Related works

4.1. DVFS scheduler implementation and evaluation with simulation

4.1.1. DVFS for periodic tasksets

DVFS techniques have been widely implemented and tested with simulations that calculate the energy using formulae with parameters derived through experience or profiling. Pillai [6] proposed a technique that effectively uses RT-DVS (Real-time Dynamic Voltage Scaling) techniques, namely the Static DVFS, CC (cycle conservative) DVFS, and LA (Look ahead) DVFS, which play an essential role in saving energy. Many DVFS algorithms have been developed for various types of tasks and different system constraint assumptions. Examples are [1,16–19].

4.1.2. DVFS for a mixed taskset with periodic/aperiodic tasks

For a Mixed Taskset scheduling that handles aperiodic requests, Lehoczky [20] developed advanced algorithms like the Deferrable Server (DS) and the Sporadic Server (SS) to handle soft deadline aperiodic tasks. The goal is to provide faster response times and still guarantee a hard deadline for periodic tasks. Digalwar [21] proposed a DVFS technique based on a Deferrable server (DS) called Energy-Efficient Dynamic Voltage Frequency Scaling (EEDVFS) that used a utilization update DVFS technique called the Cycle Conserving algorithm for handling mixed task sets. The result showed that EEDVFS reduces significant energy consumption compared to a non-DVFS approach without compromising periodic task deadlines and aperiodic task responsiveness. Sarla Mehariya [22] proposed an algorithm that uses the utilization update (Cycle conserving) DVFS technique, with a Priority Exchange Server (PES) called the Dynamic Voltage Frequency Scaling Priority Exchange Server (DVFSPE) for mixed tasks. The results of DVFSPE, when compared with EEDVFS showed significant improvement. Depending on the task set used, DVFSPE can provide up to 50% improvement in the response time of aperiodic tasks without compromising on the deadlines of the periodic task.

However, all these approaches were implemented and evaluated only in simulators and have not been evaluated in real platforms supported by real-time RTOS.

4.2. DVFS implementation in real-platforms

4.2.1. DVFS in real-platforms under linux OS

Linux Operating System provided support for DVFS governors for performance and energy tradeoff. The prebuilt DVFS (Performance, OnDemand, Conservative) Governors [23] make decisions based on CPU's load. Recent learning-based governors [24–26] are developed to learn the DVFS policies based on the history of task running. These governors are implemented independently of the schedulers in OS.

Other works perform DVFS in Linux-based real platforms based on task information. The CC algorithm and LA algorithms adjusted the frequency dynamically based on the task slack [6] and were evaluated in Hewlett-Packard N3350 notebook computer, which has an AMD K6 processor. The latest research has also shown implementation of the mixed taskset model on Linux-based environment model on multiple platforms, including Raspberry Pi and Beagle Bone platforms [15]. Energy-efficient real-time scheduling of constrained deadlines for DAG-based parallel tasks was studied on the ODROID XU-3 board for evaluation [27]. Fine-Grained per-core DVFS with per-chip DVFS was implemented and tested on three different Intel and IBM processors for various applications [28].

Although Linux provides OS-level DVFS support, it is not the best OS to choose for real-time systems.

4.2.2. Task-aware DVFS implementation in real-platforms under FreeRTOS

Due to this increasing complexity, adopting a real-time operating system to manage the interaction between tasks and their scheduling with DVFS is becoming preferable for embedded systems. FreeRTOS is one of the most widely used real-time RTOS used in the industry. Some works have implemented well-studied task-aware real-time schedulers into FreeRTOS. Kase [9] proposed the open-source Efficient Scheduling Library (ESFree) for FreeRTOS implemented in userspace that is user-friendly and runs with low overhead. Carraro [29] implemented the well-known Earliest Deadline First algorithm (EDF) and Largest Local Remaining First Execution time (LLREF) algorithm in FreeRTOS, and the proposed solutions work well according to the given specification.

However, there has not been task-aware DVFS scheduler Implementation in Real-Platforms under FreeRTOS. To the best of our knowledge, our work is the first implementation of the DVFS method for a mixed task set on a real platform powered by FreeRTOS on an ARM platform. Our implementation adopts a Deferrable Server (DS) to deal with aperiodic tasks. For the implementation and evaluation of DVFS on the FreeRTOS framework on a real platform, we choose to implement CCMT, which extends CC to deal with mixed task sets.

Table 3

Battery output shown in tera term.

	Current (mA)	Voltage (mV)	Power (uW)
Without load	7	7916	55412
With load	199	7887	1569513

5. Implementation of CCMT on FreeRTOS

Fig. 3 shows the complete system model integration of CCMT DVFS Model on the STM32 target, and each module is explained as below. The FreeRTOS platform is based on STM32-H7B31-DK hardware, and its speed ranges from 7 MHz to 280 MHz. We use a smart battery device by CED labs to power up the device and measure the voltage, current, and power levels.

We first illustrate the two hardware-related subsystems.

5.1. Frequency scaling subsystem

FreeRTOS does not provide any framework to handle frequency scaling, as in the case of the Linux platform. We implemented a customized clock driver to achieve frequency scaling on the FreeRTOS platform, providing a new framework for integrating DVFS with FreeRTOS on the STM32-H7B31-DK hardware. The customized clock driver on the STM32-H7B31-DK target is based on an RCC (Reset and Clock Control) subsystem that a number of internal or external oscillators with high or low-speed [30]. The RCC offers high flexibility for the application to switch the appropriate clock for the CPU core and peripherals ON or OFF independently. When the system is running, the user application can select the system clock (sys_ck) among the three following sources: HSE, HSI, CSI [30]. In our design, we use HSE as the source of the crystal oscillator. The functionality of the HSE is controlled by programming the RCC clock configuration register (RCC_CFGR).

5.2. Battery management system

Our implementation integrates a smart battery device (BMS — Battery Management system) module with the embedded platform. The battery management module acts as a source of power for the embedded platform and provides feedback on power consumption via the UART Terminal.

The CEDLabs Smart Battery Pack has integrated analog peripherals that act as a power source and also help us to measure and maintain a record of the available battery current, voltage, and power. The battery can also report the data to the system host controller over a Universal Asynchronous Receiver Transmitter (UART) communication [15,31].

Table 3 shows sample battery outputs without connecting to any load, and with load, respectively. The output is in the format of current (mA), voltage (mV), and power (uW).

5.3. Software handlers for CCMT

Next, we describe the software components that work under FreeRTOS to make frequency scaling decisions at scheduling points. The software components include two primary task handlers (see Fig. 4): the *Mixed_Task_Handler* and *Scheduler_Task_Handler*, plus tick ISR routines informing the arrival of tasks. The *Mixed_Task_Handler* is intended for running the user-defined tasks in the Mixed Taskset Queue. The context switch happens from the *Mixed_Task_Handler* to the *Scheduler_Task_Handler* at the scheduling point. The actual execution time is computed dynamically upon completion of the task, which is needed for calculating the frequency level. Whenever a task is completed, or a task is released, the task information in the task list and the corresponding Task Control Block (TCB) will be updated, and the frequency is calculated and selected accordingly. The software routines in the scheduling points can be found below.

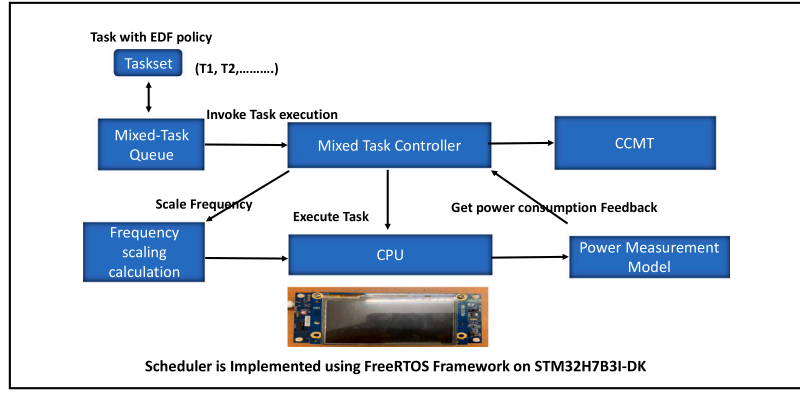


Fig. 3. CCMT system model using FreeRTOS framework.

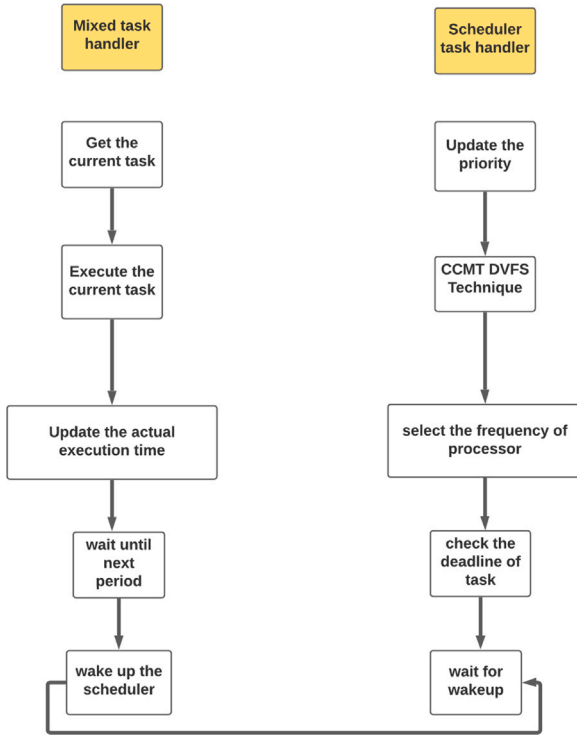


Fig. 4. Mixed_Task_Handler and Scheduler_Task_Handler on FreeRTOS.

5.4. Algorithm

Next, we show the flow chart of the algorithm.

5.4.1. Mixed_Task_Handler

The *Mixed_Task_Handler* (see Figs. 4 and 5) manages the running of periodic and aperiodic task events, updates the actual execution time of the task, and invokes the *Scheduler_Task_Handler* before and after the completion of the task, which updates the priority, invokes the CCMT algorithm and sets the frequency for the CCMT model [9].

5.4.2. Scheduler_Task_Handler

The *Scheduler_Task_Handler* is also shown in Figs. 4 and 5. The jobs get scheduled when a periodic and aperiodic task is released, completed, or preempted. The *Scheduler_Task_Handler* updates the task's priorities, considers the CCMT DVFS model, and determines the frequency the job gets scheduled. The frequency scaling is achieved by

using a customized clock driver. We scale up to five different frequencies ranging from 280 MHz to 7 Mhz. Note that the actual execution time of any periodic task or aperiodic task is always less or equal to the maximum execution time. The Tick ISR function is configured to detect the preemption of any pending task. The tick ISR hook function switches in the scheduler handler when the scheduler task period arrives. The scheduler task period in the tick ISR function can be configured by the user. The detection of deadline miss requires to scan all periodic tasks and aperiodic jobs, which has a higher execution time when compared to the deadline.

A task state (Running, Ready, Block and Suspended) in FreeRTOS is important while scheduling task [9,32]. In our implementation, we use a queue-based data structure to maintain the task details such that the highest priority task is stacked on top of the queue.

5.4.3. Context switch details

The implementation of EDF always results in a context switch to the scheduler at the beginning and end of a periodic task. The Scheduler Task will then calculate and assign priorities for every periodic task according to their absolute deadlines when it is switched in. This procedure is necessary for ensuring that the EDF policy is maintained [9]. The following are the context switch details. First, the *Mixed_Task_Handler* starts fetching the current task, Context switches (CS) to *Scheduler_Task_Handler* to update all priorities and decides the current task. Then the system makes a context switch to *Mixed_Task_Handler* to run the current task. After the current task has finished, the *Mixed_Task_Handler* updates the actual execution time. There could be task preemption which results in a task list update.

5.5. Important points to be considered while handling aperiodic task

The following are the major points that are considered during the implementation:

1. Server task occurs at every period with a budget similar to the periodic task. When the aperiodic task comes into execution, it tries to find the server budget and executes the request.
2. If the actual execution time of the aperiodic task is less than the server budget, the aperiodic task is allocated to the server budget, and the task executes.
3. We make sure that the deadline for the periodic task is not missed while serving the aperiodic task.
4. The response time of the aperiodic task is based on the actual execution time and the server budget.
5. If there are no aperiodic requests, then the overhead is the utilization of the server added to the periodic task.
6. As the number of aperiodic tasks increases, the number of jobs that the server serves increases.
7. The actual execution time for the aperiodic and periodic tasks should be considered upon completion of the task.

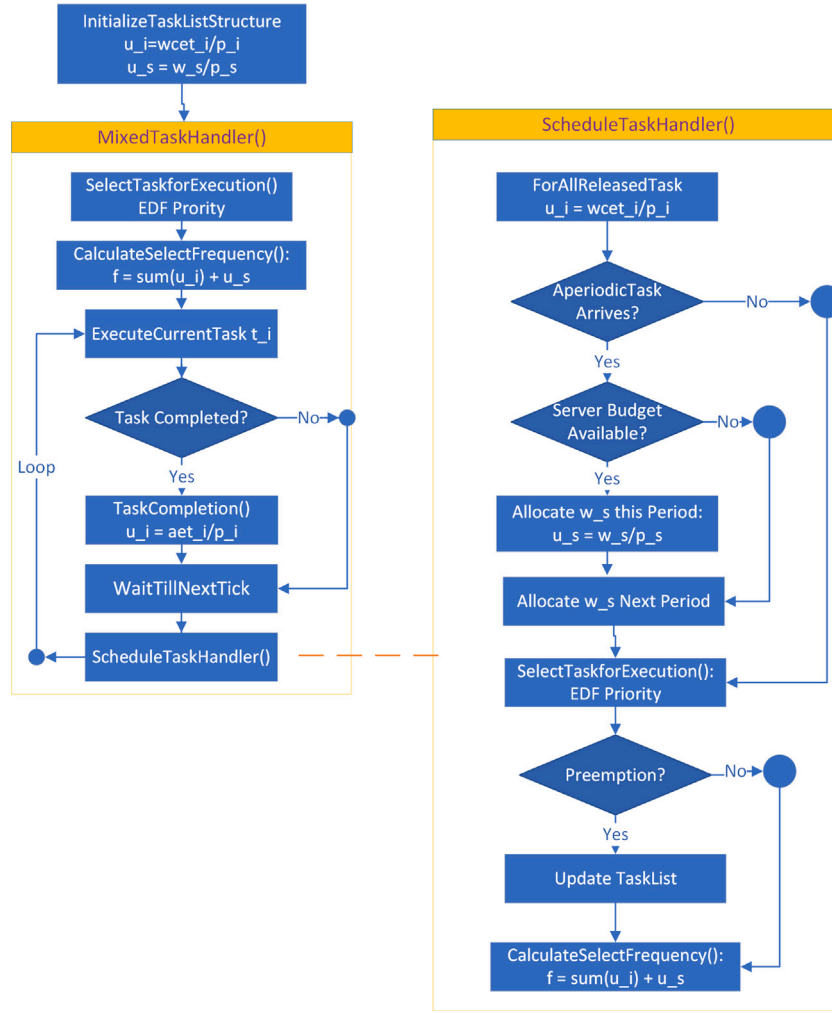


Fig. 5. Algorithm flowchart.

6. An example using CCMT scheduling

Let us consider an example that demonstrates the handling of a Mixed Taskset in the extended cycle conserving algorithm (CCMT).

The task model of a mixed task set includes the representation of periodic tasks, periodic servers, and aperiodic requests. An example is shown below.

The periodic taskset and server taskset as shown in Tables 4 and 5.

As shown in the Table 5, we allocate a server for executing the aperiodic request, and upon arrival of any aperiodic request, the budget allocated by the server task will be consumed.

In Table 6, A_1 and A_2 represent the aperiodic task request, AR is the arrival time, and AET represents the actual execution time.

Based on the server budget available, the aperiodic request gets updated in the server task. where P_n represents the server period and aet_s represents the server budget used for executing in the corresponding period.

Fig. 6 shows a CCMT example with the task execution (a), the frequency setting (b), and the task-list information at the first two scheduling points (c and d). The explanation at each scheduling point can be found below. Note that each periodic task is implemented as a FreeRTOS periodic task. The ready and non-ready task lists are implemented as lists of user-level Task Control Block (TCB) with extra fields so that we can track the actual execution time and slack for tasks loaded to the FreeRTOS platform.

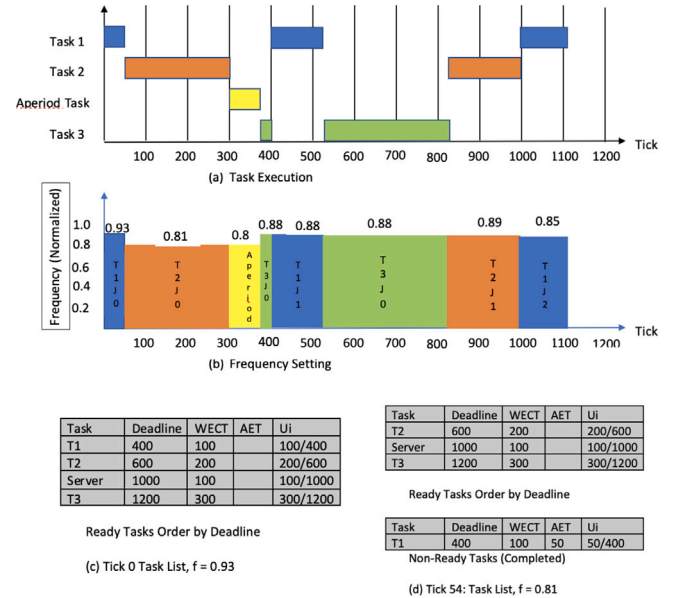


Fig. 6. Mixed taskset execution.

Algorithm 1: Extended CCMT for Mixed Taskset on FreeRTOS

```

1:  $p_i$ : Period (deadline) for task  $t_i$ 
2:  $wcet_i, aet_i$ : Worst-case Execution Time, Actual Execution Time for task  $t_i$ 
3:  $u_i$ : Utilization of task  $t_i$ 
4:  $f$ : Frequency
5: procedure INITIALIZATION();
6:   if  $\sum wcet_i/p_i > 1$  then
7:     Output Message: Taskset not schedulable and Exit
8:   foreach periodic task  $t_i$  do
9:      $u_i \leftarrow wcet_i/p_i$ 
10:     $u_s \leftarrow w_s/p_s$ 
11:    Initialize Mixed Taskset Queue Q;
12:    Add all periodic task  $t_i = (p_i, wcet_i, aet_i)$  to Q;
13:    Add aperiodic task server  $t_s = (p_s, wcet_s, aet_s)$  to Q;
14:    Initialize the priorities of the tasks in Q based on earliest deadline (EDF);
15:  Create FreeRTOS tasks: Scheduler_Task_Handler and Mixed_Task_Handler;
16:  call Mixed_Task_Handler()
17: end procedure
18: procedure MIXED_TASK_HANDLER();
19:   Select_Task_for_Execution();
20:   Calculate_Select_Frequency()
21: while do
22:   Execute the current task;
23:   if the current task  $t_i$  is completed then
24:     Compute the actual execution time  $aet_i$ ;
25:     call task_completion( $t_i$ )
26:   wait till next tick;
27:   wake_up_scheduler_task_handler();
28: end procedure
29: procedure TASK_COMPLETION( $t_i$ )
30:   update  $aet_i$  of  $t_i$ 
31:    $u_i \leftarrow aet_i/p_i$ 
32:   check the response time of aperiodic request
33:   check the deadline of periodic request
34: end procedure
35: procedure SELECT_TASK_FOR_EXECUTION()
36:   update priority of tasks in Q based on EDF;
37:   return the task  $t_i$  with the earliest deadline as the current task;
38: end procedure
39: procedure CALCULATE_SELECT_FREQUENCY()
40:   calculate  $f = \sum u_i + u_s$ ;
41:   set the suitable frequency for current task execution;
42: end procedure
43: procedure SCHEDULER_TASK_HANDLER()
44:   forall task  $t_i$  released at this tick do
45:      $u_i \leftarrow wcet_i/p_i$ ;
46:   if aperiodic task  $A_i$  arrives then
47:     if budget  $w_s$  available then
48:       allocate the budget  $w_s$ ;
49:        $u_s \leftarrow w_s/p_s$ ;
50:     else
51:       allocate the budget  $w_s$  in next server period  $p_s$ ;
52:   CurrentTask  $t \leftarrow Select\_Task\_for\_Execution()$ 
53:   if the previous active task  $t_j$  preempted by task  $t$  due to priority then
54:     update  $aet_j$  and set the current running task as  $t$ ;
55:   Calculate_Select_Frequency()
56: end procedure

```

- At time 0, considering the Taskset from Table 4, Algorithm 1 (line 7) first initializes the utilization of tasks: $u_1 = \frac{100}{400}$, $u_2 = \frac{200}{600}$, $u_3 = \frac{300}{1200}$, $u_s = \frac{100}{1000}$ (server). Based on the EDF policy, Task 1 Job 0 has the highest priority (Algorithm 1: line 14), and is chosen to execute. The frequency is

Table 4

Periodic taskset.

Task No. (T_i)	Period (P_i)	WCET (W_i)	AET (J_0)	AET (J_1)	AET (J_2)
T_1	400	100	50	100	100
T_2	600	200	200	150	200
T_3	1200	300	300	250	300

Table 5

Server taskset.

Server task	Server period (P_s)	Server budget (W_s)
S_i	1000	100

Table 6

Arrival time of an aperiodic task and execution time.

Task No.	AR	AET
A_1	200	50
A_2	600	100

Table 7

Actual execution time update in server.

	P_0	P_1	P_2	P_3
aet_s	50	100	0	0

computed based on utilization according to Algorithm 1: line 29, i.e. $f = \frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{100}{1000}$ (server) = 0.25 + 0.33 + 0.25 + 0.1 = 0.93. The detailed tasks in the ready list can be found in Fig. 6(c).

Note that normally, the CPU is not fully utilized. In this case, the utilization is 0.93 (= $\frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{100}{1000}$ (server) = 0.25 + 0.33 + 0.25 + 0.1). CCMT saves energy by setting the initial frequency to 0.93 (this frequency is set assuming all tasks execute with worst-case execution time.). In the following steps, more energy saving can be achieved by making use of the dynamic slack when tasks finish earlier ($AET < WCET$) to scale the frequency further.

Based on Table 4, the actual execution time of Task 1 Job 0 is 50 Tick. So, $(\frac{aet}{util}) \frac{50}{0.93} = 54$ Tick.

Hence, Task 1 Job 0 executes till 54 Ticks.

- At time 54 tick, Task 1 Job 0 is completed with $aet_1 = 50$, thus, $u_1 = \frac{aet_1}{p_1} = \frac{50}{400}$ (Algorithm 1: line 20).

Based on the EDF policy, Task 2 Job 0 has the highest priority (Algorithm 1: line 14) to execute, and the utilization is updated (Algorithm 1: line 29) i.e. $\frac{50}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{100}{1000}$ (server) = 0.125 + 0.33 + 0.25 + 0.1 = 0.80. The detailed tasks in the ready list can be found in Fig. 6(d). From Table 4, we find that the actual execution time of Task 2 Job 0 is 200 Tick. Hence $(\frac{aet}{util}) \frac{200}{0.80} = 250$ Ticks. So Task 2 Job 0 executes for 250 Ticks.

- At time 200 tick, an aperiodic task A_1 comes in as per Table 6. Since the deadline of the server is 1000 Tick as per Table 5, Task 2 Job 0 has the priority based on the EDF policy and continues to execute.

At time 200 Tick, the budget of the server task is checked for accommodating an aperiodic request, and updated in Table 7. Task 2 Job 0 continues and gets completed at 304 Ticks.

- At time 304 tick, task 2 Job 0 is completed, and an aperiodic request is present, which has the highest EDF priority. Hence utilization is updated (Algorithm 1: line 29) i.e. $\frac{50}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{100}{1000}$ (server) = 0.125 + 0.33 + 0.25 + 0.1 = 0.805. The actual execution time of the aperiodic task is 50 Ticks as per Table 7. So, $(\frac{aet}{util}) \frac{50}{0.805} = 63$ Tick. Here, the aperiodic task has to execute for 63 Tick, and the execution time should be 304 Tick + 63 Tick = 367 Ticks.

The current time is now updated to 367 Tick.

Effect of Variations in Number of Task

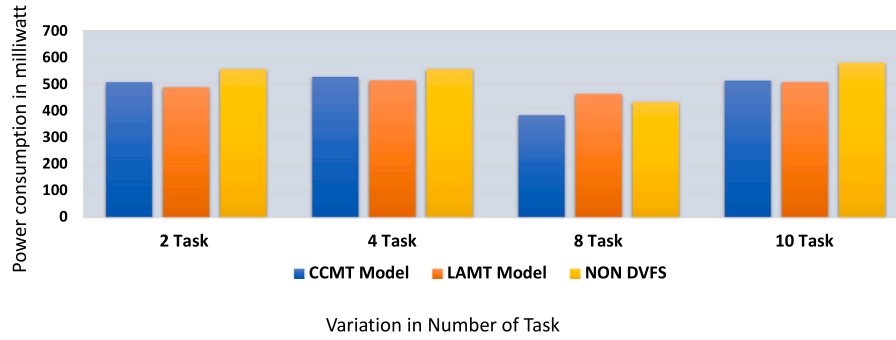


Fig. 7. Variation of Tasks in Mixed Taskset CCMT model, LAMT Model and NON-DVFS based Mixed Taskset CCMT model on STM3H7B3I-DK.

- At time 367 Tick, the aperiodic task is completed with $aet_s = 50$, thus, $u_s = \frac{aet_s}{p_s} = \frac{50}{1000}$ (Algorithm 1: line 20). The periodic request comes in and Task 3 Job 0 gets executed. The utilization is updated (Algorithm 1: line 29) i.e. $\frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{50}{1000}$ (server) = 0.125 + 0.33 + 0.25 + 0.05 = 0.88. The actual execution time of Task 3 Job 0 as per Table 4 is 300 Tick. Hence $(\frac{aet}{util}) \frac{300}{0.88} = 340$ Tick, and Task 3 Job 0 executes until 367 Tick + 33 Tick = 400 Tick.
- At time 400 Tick, The periodic task Task 1 Job 1 request is released and has highest EDF priority, and gets executed (Algorithm 1: line 14). Task 3 Job 0 is preempted. The utilization is calculated as (Algorithm 1: line 29): $\frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{50}{1000}$ (server) = 0.125 + 0.33 + 0.25 + 0.05 = 0.88. The actual execution time of Task 1 Job 1 as per Table 4 is 100 Tick. Hence $(\frac{aet}{util}) \frac{100}{0.88} = 113$ Tick, and Task 1 Job 1 executes until 400 Tick + 113 Tick = 513 Tick.
- At time 513 Tick, Task 1 Job 1 is completed with $aet_1 = 100$, thus, $u_1 = \frac{aet_1}{p_1} = \frac{100}{400}$ (Algorithm 1: line 20). Task 3 Job 0 has the priority and get selected (Algorithm 1: line 14) based on the EDF policy as per Table 4. So, utilization is updated (Algorithm 1: line 29) i.e. $\frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{50}{1000}$ (server) = 0.25 + 0.33 + 0.25 + 0.005 = 0.88. The actual execution time of Task 3 Job 0 as per left for Task 3 Job 0 is $\frac{300-33 \times 0.88}{0.88} = 307$ Tick. Thus, Task 3 Job 0 executes until time 513 Tick + 307 Tick = 820 Tick.
- At time 820 Tick, Task 3 Job 0 is completed with $aet_3 = 300$, thus, $u_3 = \frac{aet_3}{p_3} = \frac{300}{1200}$ (Algorithm 1: line 20). Task 2 Job 1 has the highest priority based on EDF policy, and the frequency is updated (Algorithm 1: line 29) i.e. $\frac{100}{400} + \frac{200}{600} + \frac{300}{1200} + \frac{50}{1000}$ (server) = 0.25 + 0.33 + 0.25 + 0.005 = 0.89. The actual execution time of Task 2 Job 1 as per Table 4 is 150 Tick. Hence $(\frac{aet}{util}) \frac{150}{0.88} = 170$ Tick. So, Task 2 Job 1 executes until 820 Tick + 170 Tick = 990 Tick.
- At time 990 Tick, Task 2 Job 1 is completed with $aet_2 = 150$, thus, $u_2 = \frac{aet_2}{p_2} = \frac{150}{600}$ (Algorithm 1: line 20). Task 1 Job 2 is chosen to execute and its frequency is calculated (Algorithm 1: line 29) i.e. $\frac{100}{400} + \frac{150}{600} + \frac{300}{1200} + \frac{100}{1000}$ (server) = 0.25 + 0.25 + 0.25 + 0.1 = 0.85. The Actual Execution time of Task 1 Job 2 as per Table 4 is 100 Tick. Hence $(\frac{aet}{util}) \frac{100}{0.85} = 117$ Tick. So, the task executes from 990 + 117 = 1107 Tick, and the deadline is met for (Task 1 Job 2) at time 1200. Thus the scheduling continues.
- The response time of the first aperiodic request is (Response Time – Arrival time): 367 Tick – 200 Tick = 167 Tick.

7. Experiment results on FreeRTOS and C++ simulator

In order to evaluate the performance of the proposed approach, we have developed a real-time environment that is capable of scheduling

real-time tasks with DVFS scaling and also a simulator version using C++. The real-time platform can configure the system to different power settings. The simulator takes a taskset consisting of only task periods, and worst-case execution time (WCET). To measure the power consumption, we have used a battery device (BMS — Battery Management system) that acts as a source of power and provides a record of the available voltage, current, and power. The experiments show the energy-saving effect of CCMT compared to the NON-DVFS model and the LAMT (Look Ahead Mixed Taskset) Model, which is based on the Look Ahead (LA) model [6].

7.1. Variation of tasks among CCMT, LAMT and NON-DVFS model

Figs. 7 and 8 show the power consumption among DVFS techniques with variations in Tasks. The taskset has both periodic and aperiodic tasks. We experimented with 2, 4, 6, 8, and 10 tasks on STM32 Platform and 5, 10, 15, 20 Tasks on C++ Simulator. WCETs in each task set are randomly chosen in the range (0.5 ms, 10 s). The AET is generated upon completion of the task and is used for further computation.

7.2. Variation of system utilization among CCMT, LAMT, and NON-DVFS model

Figs. 9 and 10 show the energy consumption among DVFS techniques with variations in system utilization on STM32 Platform and C++ Simulator. We experimented with different system utilization of 20%, 40%, 60%, and 80%.

The WCETs in each task set is randomly chosen in the range [0.5 ms, 10 s]. The AET is generated upon completion of the task and used for further computation.

Note: The taskset includes periodic and aperiodic requests.

7.3. Variation of dynamic slack tasks among CCMT, LAMT and NON-DVFS model

Figs. 11 and 12 show the energy consumption among DVFS techniques with variation in dynamic slack on STM32 Platform and C++ Simulator. We experimented with different dynamic slack 20%, 40%, 60%, and 80%.

The WCETs in each task set are randomly chosen in the range [0.5 ms, 10 s]. The AET is generated upon completion of the task and used for further computation.

Note: The taskset includes periodic and aperiodic requests.

Effect of Variation in Number of Task

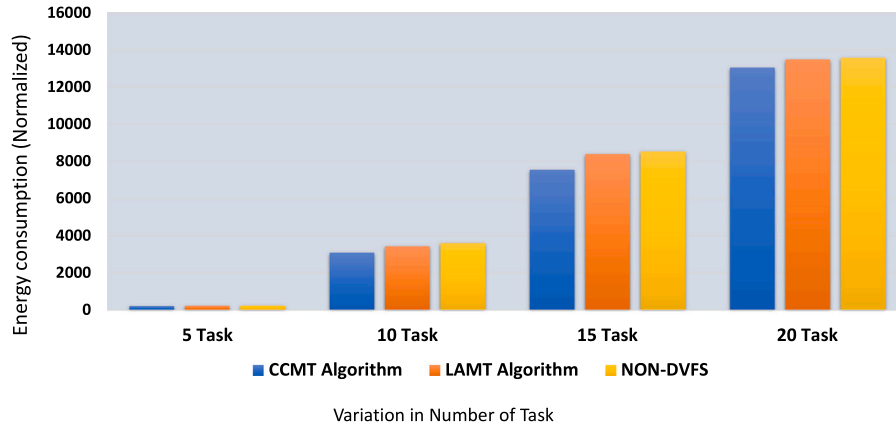


Fig. 8. Variation of Tasks in Mixed Taskset CCMT model, LAMT Model and NON-DVFS based Mixed Taskset CCMT model on C++ Simulator.

Effect of Variations in Dynamic Slack

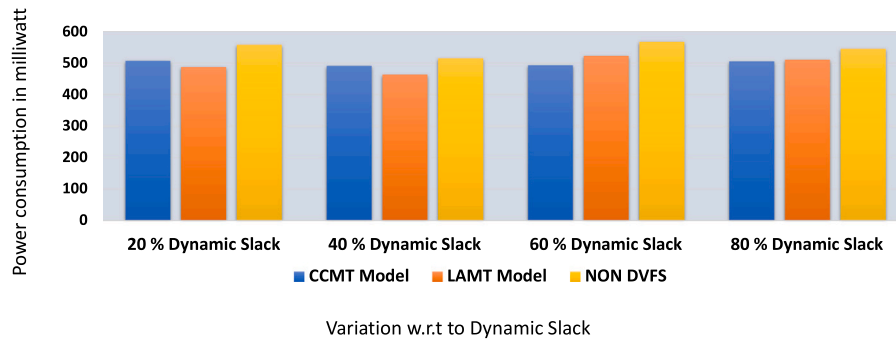


Fig. 9. Variation of system utilization in Mixed Taskset CCMT model and NON-DVFS based Mixed Taskset CCMT model on STM3H7B3I-DK.

Effect of Variation in Dynamic Slack

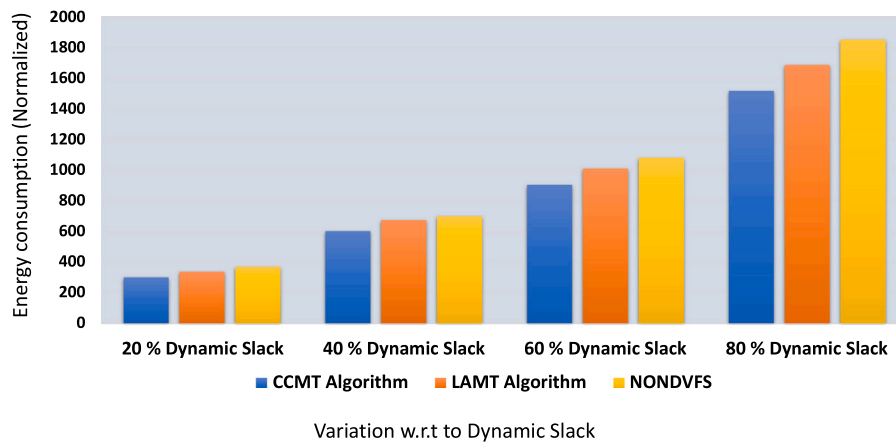


Fig. 10. Variation of system utilization in Mixed Taskset CCMT model and NON-DVFS based Mixed Taskset CCMT model on C++ Simulator.

7.4. Response time of aperiodic task

We have implemented a polling server for handling the aperiodic task, and the response time is based on the server period and server

budget. Here the service of aperiodic request is based on user intervention by pressing a button. Upon receiving the request from a user, an aperiodic request is served. Table 8 shows the polling server period and server budget, and arrival time represents the interval at

Effect of Variations in System Utilization

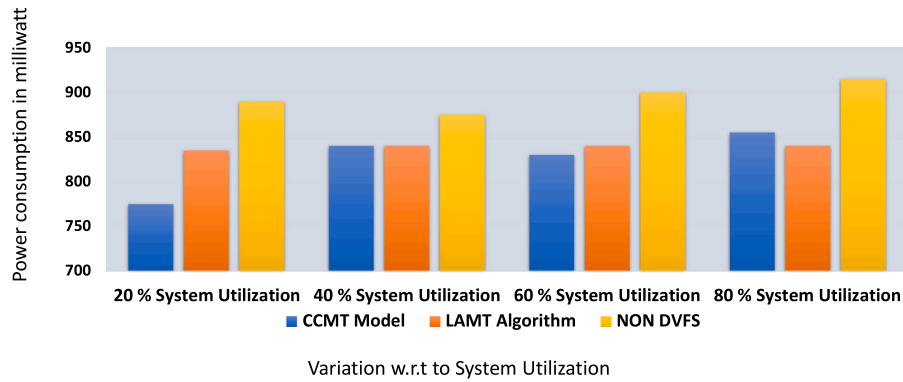


Fig. 11. Variation of Dynamic in Mixed Taskset CCMT model and NON-DVFS based Mixed Taskset CCMT model on STM3H7B3I-DK.

Effect of Variation in System Utilization

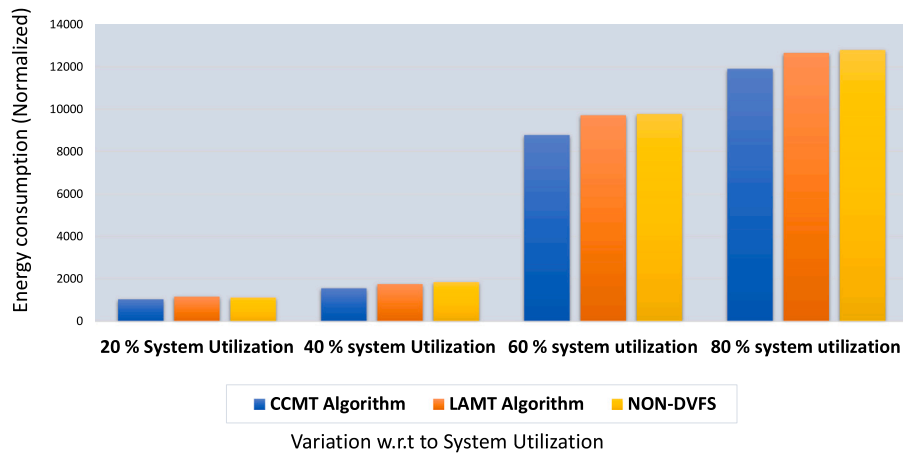


Fig. 12. Variation of Dynamic in Mixed Taskset CCMT model and NON-DVFS based Mixed Taskset CCMT model on C++ Simulator.

Table 8

Response time of aperiodic task.

Server period	Server budget	AT – CT	Response time
400 Tick	10 Tick	12802 – 12804	2 Tick
400 Tick	10 Tick	18006 – 18008	2 Tick
600 Tick	10 Tick	23423 – 23426	3 Tick
600 Tick	10 Tick	32424 – 32428	4 Tick
800 Tick	10 Tick	21607 – 21608	1 Tick
800 Tick	10 Tick	29604 – 29606	2 Tick

which an aperiodic request arrives. We have configured a simple task to be performed upon aperiodic task arrival. The response time is the difference between the arrival time and completion time.

8. Conclusion and future work

In this study, we describe the implementation and evaluation of the CCMT DVFS technique for Mixed Taskset on an embedded device powered by FreeRTOS with the goal of energy saving.

We have achieved the following:

1. We have successfully extended the CCMT technique to handle Mixed Taskset.

2. With an extra field in the user-level Task Control Block (TCB), we are able to keep track of the actual execution time and slack for tasks loaded to the FreeRTOS platform.
3. The online CCMT DVFS algorithm uses the actual execution time of the tasks and the measurement of real battery power consumption to make the decision on frequency scheduling.
4. We have implemented and evaluated the CCMT model on the real-embedded platform STM32H7B3I-DK.
5. The evaluation shows that the implemented CCMT technique in the platform described above achieves good energy savings.

For future work, we want to take application-specific task execution characteristics in the FreeRTOS kernel and extend the CCMT Model to be used in multiple applications on real-time processors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgment

The authors thank the Natural Sciences and Engineering Research Council of Canada (NSERC) for supporting this research.

References

- [1] F.M.M. u. Islam, M. Lin, Hybrid DVFS scheduling for real-time systems based on reinforcement learning, *IEEE Syst. J.* 11 (2) (2017) 931–940, <http://dx.doi.org/10.1109/JSYST.2015.2446205>.
- [2] E. Seo, J. Jeong, S. Park, J. Lee, Energy efficient scheduling of real-time tasks on multicore processors, *IEEE Trans. Parallel Distrib. Syst.* 19 (11) (2008) 1540–1552.
- [3] G. Dhiman, T.S. Rosing, System-level power management using online learning, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 28 (5) (2009) 676–689.
- [4] X. Wu, Y. Lin, J.-J. Han, J.-L. Gaudiot, Energy-efficient scheduling of real-time periodic tasks in multicore systems, in: C. Ding, Z. Shao, R. Zheng (Eds.), *Network and Parallel Computing*, in: *Lecture Notes in Computer Science*, vol. 6289, Springer Berlin Heidelberg, 2010, pp. 344–357, http://dx.doi.org/10.1007/978-3-642-15672-4_29.
- [5] H. Zeng, C.S. Ellis, A.R. Lebeck, A. Vahdat, ECOSystem: Managing energy as a first class operating system resource, *Oper. Syst. Rev.* 36 (5) (2002) 123–132.
- [6] P. Pillai, K.G. Shin, Real-time dynamic voltage scaling for low-power embedded operating systems, in: *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles*, 2001, pp. 89–102.
- [7] Q. Zhang, M. Lin, L.T. Yang, Z. Chen, S.U. Khan, P. Li, A double deep Q-learning model for energy-efficient edge scheduling, *IEEE Trans. Serv. Comput.* 12 (5) (2018) 739–749.
- [8] MaemoTM, Maemo Development, http://maemo.org/development/training/maemo_getting_started_content/plain_html/node3/.
- [9] R. Kase, Efficient Scheduling Library for FreeRTOS (Master Thesis), 2016.
- [10] D. Perale, T. Vardanega, Removing bias from the judgment day: A raven-scar-based toolbox for quantitative comparison of EDF-to-RM uniprocessor scheduling, *J. Syst. Archit.* 119 (2021) 102236, <http://dx.doi.org/10.1016/j.sysarc.2021.102236>, <https://www.sciencedirect.com/science/article/pii/S1383762121001648>.
- [11] J. Yang, G. Xu, G. Chen, N. Guan, K. Huang, Efficient runtime slack management for EDF-VD-based mixed-criticality scheduling, *J. Syst. Archit.* 117 (2021) 102119, <http://dx.doi.org/10.1016/j.sysarc.2021.102119>, <https://www.sciencedirect.com/science/article/pii/S1383762121000904>.
- [12] G.C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, third ed., Springer, New York, NY, ISBN: 978-1-4614-0675-4, 2011, <http://dx.doi.org/10.1007/978-1-4614-0675-4>.
- [13] S. Saha, B. Ravindran, An experimental evaluation of real-time DVFS scheduling algorithms, in: *Proceedings of the 5th Annual International Systems and Storage Conference*, 2012, pp. 1–12.
- [14] J.W.S. Liu, *Real-Time Systems*, Prentice Hall, Upper Saddle River, NJ, ISBN: 0130996513, 2000.
- [15] D. Ramegowda, M. Lin, Can learning-based hybrid DVFS technique adapt to different linux embedded platforms? in: 2021 IEEE SmartWorld, Ubiquitous Intelligence Computing, Advanced Trusted Computing, Scalable Computing Communications, Internet of People and Smart City Innovation, 2021, pp. 170–177.
- [16] H. Aydin, R. Melhem, D. Mossé, P. Mejía-Alvarez, Power-aware scheduling for periodic real-time tasks, *IEEE Trans. Comput.* 53 (5) (2004) 584–600.
- [17] S. Zhuravlev, J.C. Saez, S. Blagodurov, A. Fedorova, M. Prieto, Survey of energy-cognizant scheduling techniques, *IEEE Trans. Parallel Distrib. Syst.* 24 (7) (2013) 1447–1464.
- [18] X. Lin, Y. Wang, Q. Xie, M. Pedram, Task scheduling with dynamic voltage and frequency scaling for energy minimization in the mobile cloud computing environment, *IEEE Trans. Serv. Comput.* 8 (2) (2015) 175–186.
- [19] A. Saifullah, S. Fahmida, V.P. Modekurthy, N. Fisher, Z. Guo, CPU energy-aware parallel real-time scheduling, in: 32nd Euromicro Conference on Real-Time Systems, Vol. 165, ECRTS 2020, Dagstuhl, Germany, 2020, pp. 2:1–2:26.
- [20] B. Sprunt, L. Sha, J. Lehoczky, Scheduling Sporadic and Aperiodic Events in a Hard Real-Time System, Carnegie-mellon University, Pittsburgh, Software Engineering Inst. 1989.
- [21] M. Digalwar, S. Mohan, B.K. Raveendran, Dynamic voltage and frequency scaling scheduling algorithm for mixed task set, in: 2013 IEEE 8th International Conference on Industrial and Information Systems, IEEE, 2013, pp. 643–648.
- [22] S. Mehariya, D. Songara, V. Mitra, M.C. Govil, An approach for a reduced response time and energy consumption in mixed task set using a priority exchange server, in: 2014 International Conference on Advances in Computing, Communications and Informatics, ICACCI, IEEE, 2014, pp. 62–67.
- [23] D. Brodowski, N. Golde, R.J. Wysocki, V. Kumar, CPU Frequency and Voltage Scaling Code in the Linux (TM) kernel, Linux CPUFreq Governors. <https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt>.
- [24] H. Huang, M. Lin, L.T. Yang, Q. Zhang, Autonomous power management with double-q reinforcement learning method, *IEEE Trans. Ind. Inf.* 16 (3) (2020) 1938–1946, <http://dx.doi.org/10.1109/TII.2019.2953932>.
- [25] Y.-L. Chen, M.-F. Chang, C.-W. Yu, X.-Z. Chen, W.-Y. Liang, Learning-directed dynamic voltage and frequency scaling scheme with adjustable performance for single-core and multi-core embedded and mobile systems, *Sensors* 18 (2018) 3068, <http://dx.doi.org/10.3390/s18093068>.
- [26] T. Zhou, M. Lin, Deadline-aware deep-recurrent-Q-network governor for smart energy saving, *IEEE Trans. Netw. Sci. Eng.* (2021) <http://dx.doi.org/10.1109/TNSE.2021.3123280>.
- [27] A. Bhuiyan, D. Liu, A. Khan, A. Saifullah, N. Guan, Z. Guo, Energy-efficient parallel real-time scheduling on clustered multi-core, *IEEE Trans. Parallel Distrib. Syst.* 31 (9) (2020) 2097–2111, <http://dx.doi.org/10.1109/TPDS.2020.2985701>.
- [28] B. Acun, K. Chandrasekar, L.V. Kale, Fine-grained energy efficiency using per-core DVFS with an adaptive runtime system, in: 2019 Tenth International Green and Sustainable Computing Conference, IGSC, 2019, pp. 1–8, <http://dx.doi.org/10.1109/IGSC48788.2019.8957174>.
- [29] E. Carraro, Implementation and Test of EDF and LLREF Schedulers in FreeRTOS, (Thesis), University of London, 2016.
- [30] S.M. Electronics, RM0455 Reference Manual STM32H7A3/7B3 and STM32H7B0 Value line Advanced Arm-based 32-bit MCUs. https://www.st.com/resource/en/reference_manual/rm0455-stm32h7a37b3-and-stm32h7b0-value-line-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [31] Agnion Lithium Ion Smart Battery with I2C Communication and CEDLabs, <http://www.cedlabs.in/Battery.php>.
- [32] Mastering the FreeRTOS Real-Time Kernel, accessed on 05.10.2021. https://freertos.org/Documentation/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf.



Deepak Ramegowda is currently working in Qualcomm Canada. He graduated from St Francis Xavier University under the guidance of Dr. Man Lin. His research interests include Deep-Learning in Embedded Systems, Linux, and Real-time Operating Systems.



Man Lin received her B.E. degree in Computer Science and Technology from Tsinghua University, Beijing, China, and her Ph.D. degree from Linköping University, Sweden. She is currently a Professor in Computer Science at St. Francis Xavier University, Canada. Her research interests include Cyber-physical Systems, Real-Time Scheduling, Power-aware Computing, Machine Learning, System Analysis, and Optimization Algorithms.